
Post-Quantum Secure Communication Architecture for Industrial Robotic Production Cells

Software Architecture Documentation

Version 1.0 — May 2026

Document Version	1.0
Date	23/05/2026
Classification	Competition Submission
Competition	Airbus Fly Your Ideas 2026 — Round 2
Team Members	Anusha Jain, Jignya Kadali, Krish Dhankar, Nikhil Reddy
Institution	ISAE-SUPAERO, Toulouse, France

Contents

1	Purpose	2
2	System Overview	2
3	File Structure	2
4	Attack Detection Summary	4
5	Performance Metrics	5
6	System Demonstration	5

1 Purpose

The purpose of this document is to highlight the technical overview of the software implementation of the prototype. It describes the structure and purpose of each module in the codebase. The system is fully implemented and operational; all components described in this document correspond to working code that can be run and demonstrated live.

2 System Overview

The platform secures OT-to-IT telemetry in industrial robotic production cells using post-quantum cryptography. The system has three layers: the **OT Layer** (ESP32 sensor node + OT Signer) generates and signs telemetry; the **GA Layer** (Global Aggregator, CA, device provisioning, crypto utilities) verifies, encrypts, and routes packets; the **IT Layer** (visualize + dashboard) decrypts and monitors data in real time. A separate **adversarial test suite** (`attacks.py`) validates all six attack detection capabilities. All cryptographic operations use NIST-standardized ML-DSA-44 (Dilithium) and ML-KEM-512 (Kyber).



3 File Structure

OT Layer — ESP32 Device & OT Signer

File	Purpose	Inputs / Outputs	Role
<code>esp32_sender_hardware.ino</code>	Runs on the ESP32. Reads sensor data, controls the servo and actuator based on object proximity, and sends the readings as a telemetry packet to the OT Signer every 2 seconds.	In: sensor readings Out: telemetry packet → OT Signer	<i>OT sensor</i>
<code>ot_signer.py</code>	Receives raw telemetry from the ESP32, digitally signs it to prove it came from a trusted device, attaches the device certificate, and forwards the authenticated packet to the GA layer.	In: raw telemetry Out: signed packet → GA layer	<i>Packet signing & forwarding</i>

GA Layer — Global Aggregator · CA · Device Setup · Crypto Utilities

File	Purpose	Inputs / Outputs	Role
aggregator.py	The main security checkpoint. Verifies every packet's certificate and signature, and rejects replayed or delayed packets. Approved packets are re-encrypted using Kyber key exchange, HKDF key derivation and AES-GCM before forwarding to the IT layer. Logs live metrics to <code>metrics.json</code> .	In: signed packet Out: encrypted packet → IT layer	<i>Central security verifier</i>
ca.py	Run once at setup to create the Certificate Authority. Generates the root cryptographic keypair that the rest of the system trusts. The public key is shared with the GA so it can verify that device certificates are genuine.	In: — Out: CA public & private keys	<i>Certificate Authority</i>
device_setup.py	Run once per device to register it on the network. Generates a unique cryptographic identity for the OT device and obtains a CA-signed certificate to verify that the device is trusted. The device then uses this certificate to authenticate itself while transmitting telemetry data.	In: CA private key Out: device certificate & private key	<i>Device identity provisioning</i>
crypto_utils.py	A shared utility library that provides all cryptographic functions used across the system — generating keys, signing and verifying messages using Dilithium (ML-DSA-44), and performing key exchange using Kyber (ML-KEM-512). Keeps all cryptographic logic in one place so other modules do not repeat it.	In: keys / messages Out: signatures / shared secrets	<i>PQC library</i>

- **ML-KEM-512 (Kyber):** Performed once at the start of each session to establish a secure post-quantum shared secret between the GA and IT layers using the IT public key.
- **HKDF (HMAC-based Key Derivation Function):** Derives a unique AES-256 traffic key for each telemetry packet using the shared secret and packet counter.
- **AES-GCM:** Uses the derived AES keys to securely encrypt and authenticate telemetry before forwarding it to the IT layer.

IT Layer — Secure Decryption & Real-Time Monitoring

File	Purpose	Inputs / Outputs	Role
visualize.py	Decrypts incoming packets using Kyber key exchange, HKDF key derivation and AES-GCM, recovering the original OT telemetry at the IT layer.	In: encrypted packet Out: decrypted telemetry	<i>Secure telemetry decryption</i>
dashboard.py	A Streamlit web dashboard that displays the security status of the system — accepted and blocked packet counts, average latency, memory usage and any detected attacks. Updates every 2 seconds so operators can monitor the OT network in real time.	In: metrics.json, attack_history.json Out: real-time web dashboard	<i>Live security dashboard</i>

Adversarial Test Suite — attacks.py

File	Purpose	Attack classes	Role
attacks.py	A test tool that simulates six types of attack against the GA layer to validate that the security checks work correctly. Each attack is sent as a real packet and blocked by the GA within one cycle. The active attack is logged for the dashboard to display.	Replay, Delay, Payload modification, Signature corruption, Fake certificate, Unauthorized device	<i>Attack simulation & validation</i>

4 Attack Detection Summary

Attack	Type	How it is detected	GA response
Replay	Replay	Packet counter must strictly increase per device	BLOCKED: replay attack
Delay	Replay	Packet rejected if timestamp is older than 5 s	BLOCKED: stale packet
Payload modification	Modified	Any change to payload invalidates the Dilithium signature	INVALID DILITHIUM SIGNATURE
Signature corruption	Modified	Dilithium verification fails on non-matching signature bytes	INVALID DILITHIUM SIGNATURE
Fake certificate	Fabricated	Certificate signature does not match the trusted CA public key	FAKE CERTIFICATE
Unauthorized device	Unauthorized	Device keypair not signed by the CA — certificate rejected	UNAUTHORIZED DEVICE

5 Performance Metrics

Metric	What it represents
Average end-to-end latency	The average time between a packet being sent by the ESP32 and the decrypted telemetry arriving at the IT layer. Covers the full pipeline including Wi-Fi transmission, signing, verification and re-encryption.
Verification runtime	The time taken by the GA to verify the Dilithium signature on each incoming packet. Confirms that post-quantum authentication adds negligible overhead to the telemetry pipeline.
Memory overhead	The RAM used by the aggregator process during live operation. Confirms that the full PQC stack runs comfortably on a standard laptop or edge server without dedicated hardware.
Valid packets	The count of packets that passed all three security checks — certificate validity, payload signature, and timestamp and counter freshness — and were successfully forwarded to the IT layer.
Blocked packets	The count of packets rejected by the GA due to a failed security check. Each blocked packet corresponds to a detected attack or an invalid transmission.
Success rate	The proportion of received packets that were authenticated and accepted. Calculated as valid packets divided by total packets received, expressed as a percentage.

6 System Demonstration

The demonstration runs as a live system on a local network. The ESP32 continuously transmits real sensor telemetry, which flows through the OT Signer, Global Aggregator and IT layer in real time. The Streamlit dashboard is visible throughout, showing live packet counts, latency and system status.

To demonstrate security, the user selects any of the six attacks from `attacks.py`. The GA detects and blocks the attack within one packet cycle, and the dashboard immediately updates to show the attack category, the reason it was blocked, and the protection mechanism that caught it. All six attack classes can be demonstrated in sequence without restarting the system.